1. (a) What is the arithmetic intensity of matrix addition $C = A + B$ where A, B, and C are $N \times N$ matrices with FP32 elements? [1 pt]

   (b) You are running an ML inference model requiring a throughput of 20 images/sec. On your current machine with a 1.5 GHz processor, using SIMD and multi-core optimizations, you achieve only 15 images/sec. To reach the required throughput, you plan to buy a machine with the same specs but a 3 GHz processor. Will this solve your problem? Justify your answer. [1 pt]

   (c) Your matrices are stored in column-major fashion. For matrix multiplication, between the following two loop orderings, which one performs better? [2 pts]

```
// ikj
for (i=0; i<N; i++)
  for (k=0; k<N; k++)
    for (j=0; j<N; j++)
      C[i][j] += A[i][k]*B[k][j];
```

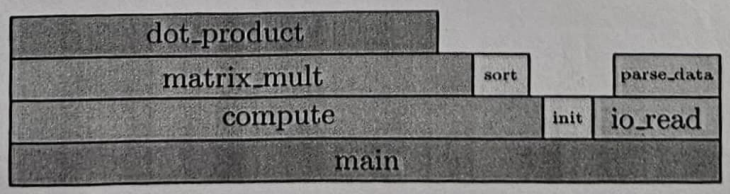
```
// jki
for (j=0; j<N; j++)
  for (k=0; k<N; k++)
    for (i=0; i<N; i++)
      C[i][j] += A[i][k]*B[k][j];
```

   (d) You need to sort 1 TB of data on a machine with 64 GB RAM. Some ways to improve overall sorting time include: (i) Increase RAM to 128 GB, (ii) Use a more efficient sorting algorithm, (iii) Compress intermediate data. Suggest 2 additional valid ways to improve sorting time. [2 pts]

2. For each characteristic below, mark which profiling tools it applies to. **Fill the table with ✓ for applicable and ✗ for not applicable. Each row carries 0.5 marks; all 5 entries in a row must be correct.** [3½ pts]

S.No	Characteristic	gprof	PAPI	perf	Valgrind	Intel PIN
1	Requires recompilation					
2	Requires source code modification					
3	Works on unmodified binaries					
4	Uses hardware performance counters					
5	Dynamic binary instrumentation					
6	Simulates CPU execution (very high overhead)					
7	Sampling-based profiling					

3. Consider the following flame graph generated from profiling a program. Answer the questions below (0.5 marks each). [1½ pts]



(i) Which function is the performance bottleneck in this program?

(ii) Is `dot_product()` called by `matrix_mult()` only once?

(iii) Is `matrix_mult` coming before `sort` because it takes more time to execute?

4. You have a query string which you want to compare against a database of strings on a single-core machine. The comparison algorithm involves computing a dynamic programming (DP) matrix. What would be your SIMD strategy to accelerate this computation? Also mention a couple of challenges you may face in your approach. [2 pts]

5. For each of the following loops, state whether it is vectorizable or not. Justify your answer in 1-2 lines. (1 mark each) [3 pts]

(a)
```
for (int i = 1; i < N; i++)
    A[i] = A[i-1] * 2;
```

(b)
```
for (int i = 0; i < N; i++)
    A[idx[i]] = B[i];
```

(c)
```
for (int i = 0; i < N; i++)
    if (A[i] > 0) B[i] = A[i] * 2;
```

6. (a) Consider the following triangular loop. Which OpenMP scheduling strategy (static, dynamic, or guided) would you choose? Justify your answer. [1½ pts]

```
#pragma omp parallel for schedule(???)
for (int i = 0; i < N; i++) {
    for (int j = 0; j < i; j++) {
        C[i][j] = A[i][j] + B[i][j];
    }
}
```

(b) Consider the following code with cyclic scheduling. What performance issue can arise? How would you fix it? [1½ pts]

```
int results[N];
#pragma omp parallel for schedule(static, 1)
for (int i = 0; i < N; i++) {
    results[i] = compute(i);
}
```

7. To perform convolution using im2col, we compute $Y = W \times X_{col}$. Given:

- Input image: 4×4
- 3 filters (F_1, F_2, F_3), each 2×2
- Stride = 1

```
Input (4x4):
a  b  c  d
e  f  g  h
i  j  k  l
m  n  o  p
```

```
Filters (2x2 each):
F1: [r1 r2]   F2: [b1 b2]   F3: [g1 g2]
    [r3 r4]       [b3 b4]       [g3 g4]
```

(a) What are the dimensions of W and X_{col}? [1 pt]

(b) Write the W matrix. [1 pt]

(c) Write the first 2 columns and last 2 columns of X_{col}. [1 pt]

(d) What is the memory blowup factor for X_{col} compared to the original input? [1 pt]

8. The following code uses OpenMP tasks to process a linked list, but it produces incorrect results. Identify the bug and fix it. [2 pts]

```
void process_list(Node* head) {
    Node* curr = head;
    #pragma omp parallel
    {
        #pragma omp single
        {
            while (curr != NULL) {
                #pragma omp task
                {
                    process(curr->data);
                }
                curr = curr->next;
            }
        }
    }
}
```